Welcome to the 2nd semester in MCA

Happy New Year

"**Never** stop Learning, **Because** life never stops teaching"

UNIT 1:
FUNDAMENTALS OF CLOUD COMPUTING

AGENDA

- Definition
- Cloud Service Models
- Cloud Deployment Models
- Advantages and Challenges of Cloud Computing
- Introduction to Google Cloud Platform (GCP)

INTRODUCTION

What is Cloud?

The term **Cloud** refers to a **Network** or **Internet**.

In other words, we can say that Cloud id something,

Which is present at remote location.

Cloud can provide service over network, i.e.,

On public networks or on private networks, i.e.,

WAN, LAN or VPN.

Applications such as **e-mail, web conferencing, customer relationship management (CRM),** all run in cloud.

DEFINITIONS

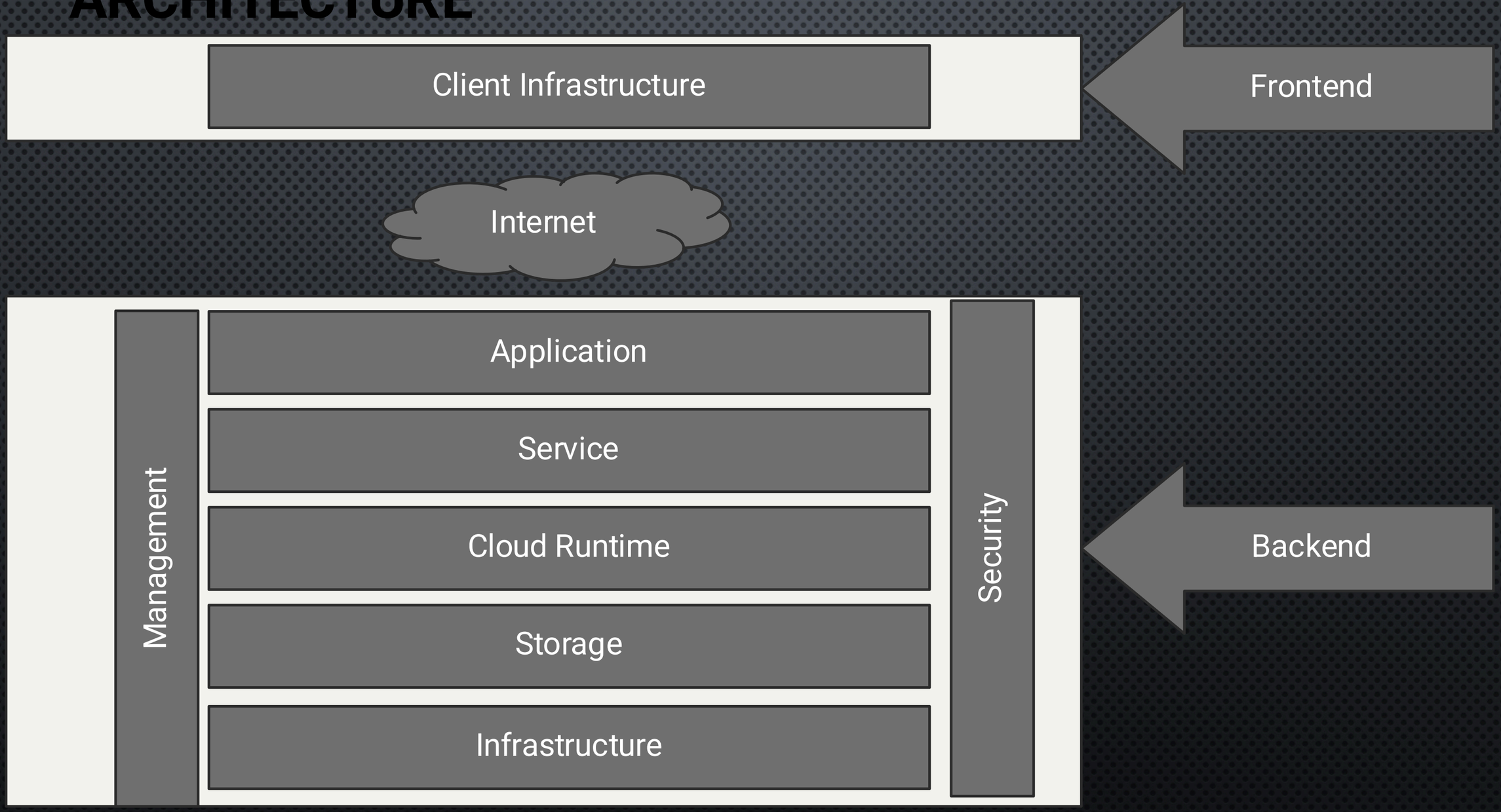
What is Cloud Computing?

Cloud Computing refers to manipulating, configuring, and accessing the applications online.

It offers online data storage, infrastructure and application.

Cloud Computing is both a combination of software and hardware based computing resources delivered as a network service.

ARCHITECTURE



BASIC CONCEPTS

There are certain services and models working behind the scene making the cloud computing feasible and accessible to end users. Following are the working models for cloud computing:

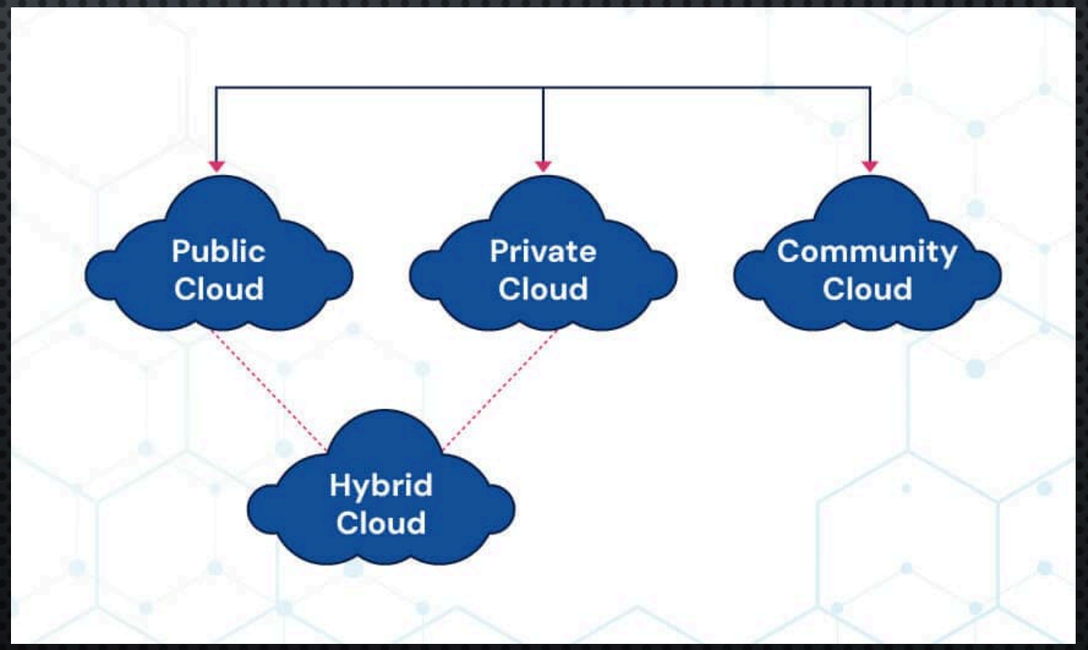
1. Cloud Deployment Model
2. Cloud Service Model

CLOUD DEPLOYMENT MODELS

Deployment models define the type of access to the cloud, i.e., how the cloud is located?

Cloud can have any of the four types of access:

- Public Cloud,
- Private Cloud,
- Hybrid Cloud and
- Community Cloud.



CLOUD DEPLOYMENT MODELS

Public Cloud: The Public cloud allows systems and services to be easily accessible to the general public. Public cloud may be less secure because of its openness, e.g., e-mail.

Private Cloud: The Private Cloud allows systems and services to be accessible within an organization. It offers increased security because of its private nature.

CLOUD DEPLOYMENT MODELS

Hybrid Cloud: The Hybrid Cloud is a mixture of public and private cloud. However, the critical activities are performed using the private cloud while the non-critical activities are performed using the public cloud.

Community Cloud: The Community Cloud allows systems and services to be accessible by a group of organizations.

CLOUD SERVICE MODELS

Service Models are the reference models on which the Cloud Computing is based. These can be categorized into three basic service models as listed below:

1. Infrastructure as a Service (IaaS)
2. Platform as a Service(PaaS)
3. Software as a Service (SaaS)
4. Anything/Everything as a Service (XaaS)

INFRASTRUCTURE AS A SERVICE (IAAS)

IaaS is the delivery of technology infrastructure as an on demand scalable service.

IaaS provides access to fundamental resources such as physical machines, virtual machines, virtual storage, etc.

- Usually billed based on usage
- Usually, multi tenant virtualized environment
- Can be coupled with Managed Services for OS and application support

INFRASTRUCTURE AS A SERVICE (IAAS)

Examples

GO GRID

NTT Communications

BLIZZARD ENTERTAINMENT

OpSource

terremark

at&t ECT

PLATFORM AS A SERVICE (PAAS)

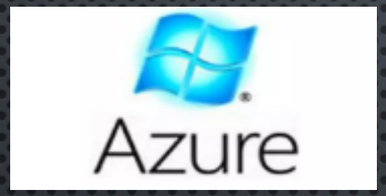
PaaS provides the runtime environment for applications, development & deployment tools, etc.

PaaS provides all of the facilities required to support the complete life cycle of building and delivering web applications and services entirely from the Internet. Typically, applications must be developed with a particular platform in mind.

- Multi tenant environments
- Highly scalable multi tier architecture

PLATFORM AS A SERVICE (IAAS)

Examples



SOFTWARE AS A SERVICE (PAAS)

SaaS model allows to use software applications as a service to end users.

SaaS is a software delivery methodology that provides licensed multi-tenant access to software and its functions remotely as a Web-based service.

- Usually billed based on usage
- Usually multi-tenant environment
- Highly scalable architecture

SOFTWARE AS A SERVICE (IAAS)

Examples

BENEFITS OF CLOUD COMPUTING

Cost Efficiency: Reduces upfront hardware/software investment, shifting to operational expenses (pay-as-you-go).

Scalability & Flexibility: Easily scale resources up or down with demand, avoiding over-provisioning.

Accessibility & Mobility: Access data and applications from any device, anywhere with an internet connection, enabling remote work.

Automatic Updates & Maintenance: Providers manage infrastructure updates, patches, and maintenance.

BENEFITS OF CLOUD COMPUTING

Disaster Recovery: Built-in redundancy and backups improve data security and business continuity.

Collaboration: Facilitates real-time sharing and collaboration on documents and applications.

Innovation: Access to advanced technologies like AI/ML and faster deployment of new services.

CHALLENGES OF CLOUD COMPUTING

Security & Privacy: Shared responsibility model means misconfigurations can lead to breaches; data privacy concerns remain.

Internet Dependency: Requires reliable internet; outages can disrupt access.

Compliance & Regulations: Meeting industry-specific data residency and regulatory requirements can be complex.

Limited Control: Less direct control over underlying infrastructure compared to on-premises systems.

Cost Management: While cost-effective, costs can escalate if usage isn't monitored (cost optimization is key).

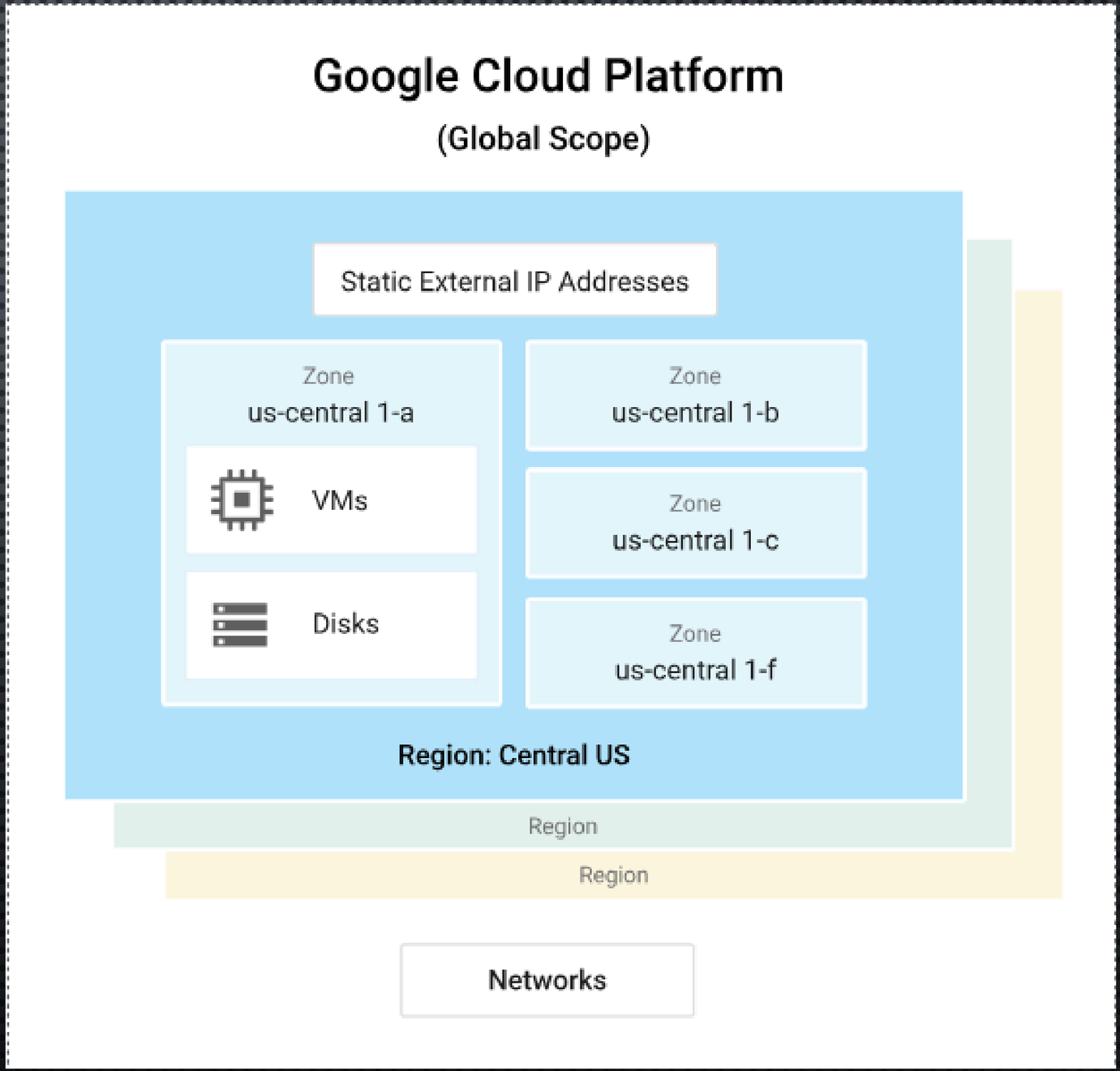
INTRODUCTION TO GOOGLE CLOUD PLATFORM (GCP)

Google Cloud Platform enables developers to build, test and deploy applications on Google’ s highly scalable and reliable infrastructure. Choose from computing storage and application services from your web, mobile and Backend solutions.

GCP is a set of modular cloud-based services that allow you to create anything from simple to complex applications.

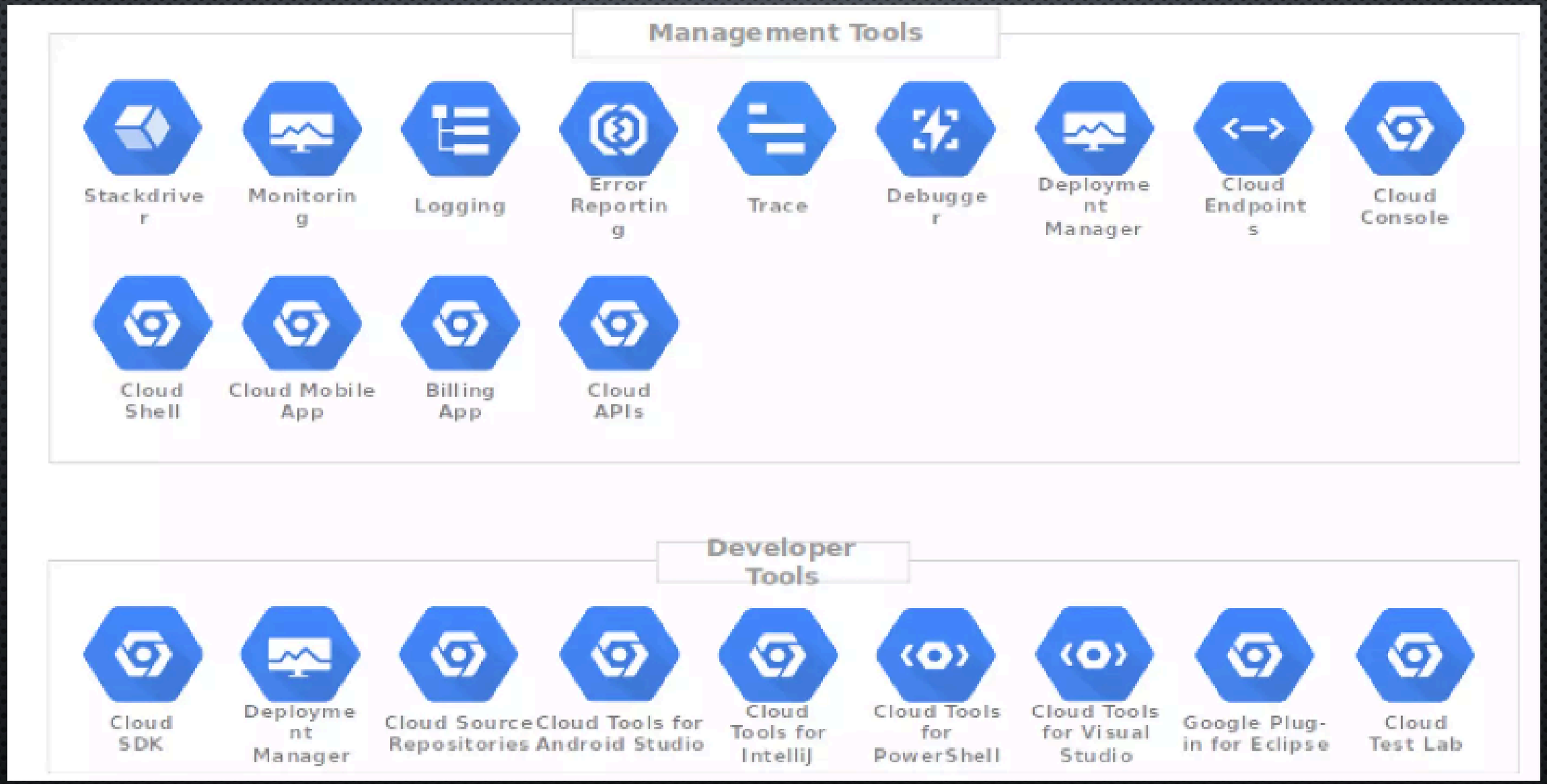
These services/resources are categorized as Global, regional, or Zonal resources

- Global resources include preconfigured disk images, disk snapshots, and networks
- Regional resources include static external IP addresses
- Zonal resources include VM instances, their types, and disks



SERVICES PROVIDED BY GCP

SERVICES PROVIDED BY GCP



CLOUD IAM

- With Cloud IAM, you manage access control by defining who (identity) has what access (role) for which resource.
- Enterprise-grade access control
- Features:
  - Single access control interface
  - Fine-grained control
  - Context-aware access
  - Web, programmatic, and command-line access
  - Free of charge

ACCESS MANAGEMENT HAS THREE MAIN PARTS:

- **Member:** A member can be a Google Account (for end users), a service account (for apps and virtual machines), a Google group, or Cloud Identity domain that can access a resource.
- **Role:** A role is a collection of permissions. Permissions determine what operations are allowed on a resource.
- **Policy:** The Cloud IAM policy binds one or more members to a role.

SERVICE ACCOUNT

- A service account is an account for an application instead of an individual end user.
- When you run code that’ s hosted on Google Cloud, the code runs as the account you specify.
- You can create as many service accounts as needed to represent the different logical components of your application

COMPUTE ENGINE

- Scalable and high-performance virtual machines
- Features:
  - Predefine machine types
  - Custom machine types
  - Preemptible VMs
  - Live migration for VMs
  - Per-second billing
  - Sustained-use savings

CLOUD STORAGE

- Unified object storage for developers and enterprises
- Features:
  - Storage classes for any workload
  - Lacation types for different redundancy and performance needs
  - Object lifecycle management
  - Retention policies
  - Customer-supplied encryption keys
  - Pub/Sub Notifications for Cloud Storage
  - Requester pays

VIRTUAL PRIVATE CLOUD

- A private network within Google Cloud Platform infrastructure
- Features:
  - VPC network
  - Packet Mirroring
  - VPN
  - VPC peering
  - Shared VPC
  - Bring your own Ips
  - Disaster recovery

- **Laboratory Task:**

  Access the Google Cloud Console using the URL

  https://console.cloud.google.com/. Explore the Google Cloud Platform to

  understand its interface, available services, and overall functionality.

Unit 2:
Fundamentals of DevOps

2.1. DevOps: Definition, Concepts and Benefits

2.2. DevOps Lifecycle: CI/CD, Automation, and Monitoring

2.3. DevOps Best Practices (Agile, Automation, Release Management)

2.4. DevOps Tools overview

2.5. Overview of GCP Services for DevOps

DEFINITION

- DevOps is a software development methodology that accelerates the delivery of high-performance applications and services by combining and automating the work of software development (Dev) and IT operations (Ops) teams.

  - IBM

- DevOps is a modern approach to software development that brings development and operations teams together to deliver applications faster and more reliably. It focuses on collaboration, automation, and continuous improvement across the software lifecycle.

CONCEPTS

- Automation: The backbone of DevOps, used to perform repetitive tasks such as builds, testing, and deployments without manual intervention to reduce human error.
- Continuous Integration and Continuous Delivery (CI/CD): Automates the process of merging code changes frequently, running tests, and preparing builds for immediate release to production.
- Infrastructure as Code (IaC): Managing and provisioning infrastructure using descriptive coding languages instead of manual configurations, ensuring consistency across environments.
- Monitoring and Observability: Continuous tracking of application performance and system health using real-time telemetry, logs, and alerts to resolve issues proactively.

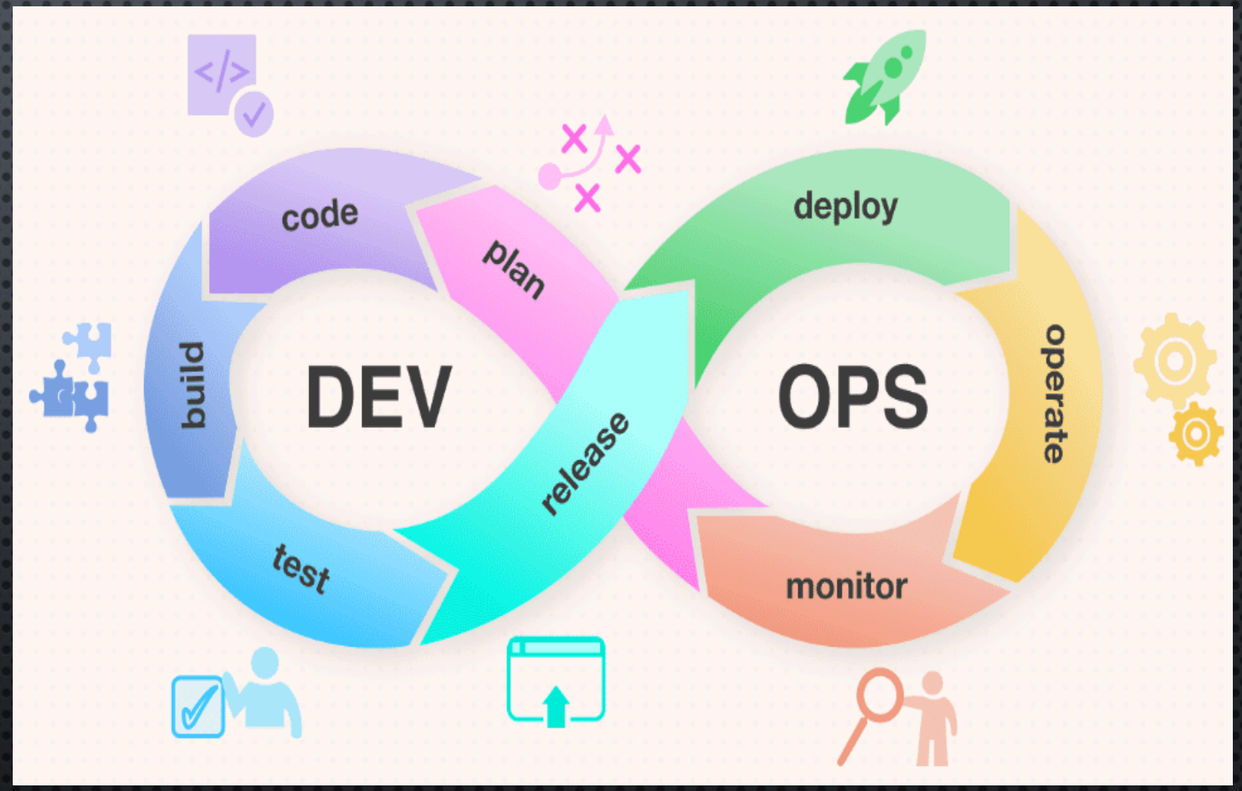
KEY BENEFITS

- Faster Time to Market: Frequent, smaller updates allow organizations to innovate and respond to customer needs more quickly than traditional models.
- Improved Software Quality: Automated testing and continuous feedback loops significantly reduce the number of bugs reaching production.
- Higher Reliability and Stability: Consistent environments created via IaC and rapid recovery through automated rollbacks lead to improved system uptime.
- Reduced Costs: Automation minimizes manual toil and rework, while optimized resource management (often through cloud-native tools) lowers infrastructure expenses.
- Enhanced Security: By integrating security into the pipeline (DevSecOps), compliance is enforced automatically, reducing the risk of breaches

LIFE CYCLE

DEV

- **Plan:** Collaborate with stakeholders to define project scope, features, and testing goals. Estimate timelines for development and testing phases.
- **Code:** Write clean, well-tested code based on the plan. Utilize version control systems (like Git) for code management. Integrate unit tests to ensure individual code functionality.



- **Build:** Automate the build process using CI/CD pipelines. Integrate tools to compile and package code into a deployable format.
- **Test:** Conduct unit tests written during coding. Collaborate with QA to define and execute manual and automated tests. Identify and fix bugs before deployment.

OPS

- **Release**: Package the application code for deployment. Manage release cycles and create release versions. Communicate release plans to stakeholders.
- **Deploy**: Utilize CI/CD pipelines to automate deployments. Deploy the application to production servers with minimal downtime. Roll back deployments if issues arise.
- **Monitor**: Set up monitoring tools to track application performance and health. Analyze logs and metrics to identify potential problems. Receive alerts for critical issues requiring intervention.
- **Operate**: Manage the production environment, including servers and infrastructure. Scale resources as needed to handle traffic fluctuations. Perform maintenance tasks to ensure smooth application operation. Respond to incidents and troubleshoot application issues.

DEVOPS TOOLS

https://www.atlassian.com/devops/devops-tools

Jira

kubernetes

GitLab

docker

Jenkins

Bitbucket

GitHub

DEVOPS BEST PRACTICES (AGILE, AUTOMATION, RELEASE MANAGEMENT)

https://www.atlassian.com/devops/what-is-devops/devops-best-practices

1. Agile project management

Agile is an iterative approach to project management and software development that helps teams deliver value to their customers faster and with fewer headaches. Agile teams focus on delivering work in smaller increments, instead of waiting for a single massive release date. Requirements, plans, and results are evaluated continuously, allowing teams to respond to feedback and pivot as necessary.

4. Implement automation

Continuous integration and delivery allows developers to merge code regularly into the main repository. Instead of manually checking code, CI/CD automates this process, from batching in a specified window to frequent commits. In addition to CI/CD, automated testing is essential to successful DevOps practices. Automated tests might include end-to-end testing, unit tests, integration tests, and performance tests. Read more about incorporating automation into your software development processes. Read more about automation.

Unit 3: Containerization with Docker and Kubernetes

3.1. Containerization and Docker: Overview, Benefits

3.2. Running Docker Containers

3.3. Introduction to Kubernetes

3.4. Deploying a Simple Kubernetes Application on GCP

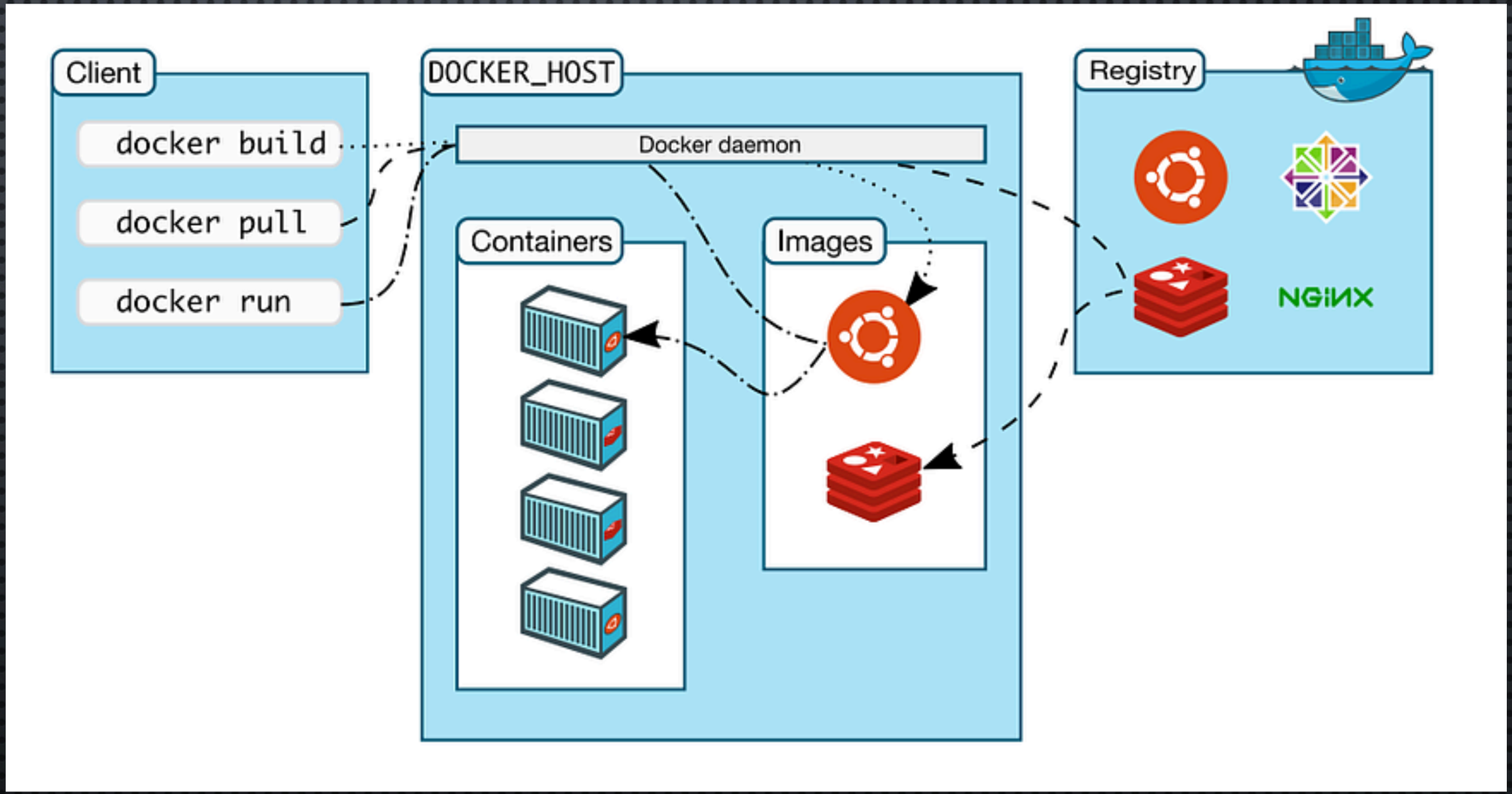
docker kubernetes

WHAT IS DOCKER?

- Docker is an open platform for developing, shipping, and running applications. It is designed to deliver your applications faster. With docker you can separate your applications from your infrastructure and treat your infrastructure like a managed application.
- **Features of Docker:**
  - Extremely fast and elegant isolation framework
  - Inexpensive
  - Low CPU/memory overhead
  - Fast boot/shutdown
  - Cross cloud infrastructure

DOCKER COMPONENTS AND ARCHITECTURE

- **Components:**
  - Docker Client
  - Docker Daemon
  - Docker Index
  - Docker Containers
  - Docker Images
  - Dockerfile



RUN DOCKER CONTAINER

Step 1: Build an Image.

Step 2: Run the Container.

Pulling a Docker Image:

1. Go to the Docker Hub website [https://hub.docker.com/]
2. Searching an images "hello world" .
3. You will find the command to run the image:

   docker run hello-world

4. Copy the command and past in your terminal

Run the Container:

1. Now image convert into the container
2. Put this command in terminal:

   docker run hello-world

3. Now stop docker running command:

docker stop <container ID>

Docker run – it ubuntu

WHAT IS K8S?

- Kubernetes (K8S) is an open-source tool for managing containerized workloads.
- It operated at the container (not hardware) level to automate the deployment, scaling and management of applications.
- K8S works alongside a containerization tool, like Docker. So, if containers are the "ingredients" of an application, then K8S would be the "Chef".
- Features of K8s
  - Horizontal Scaling
  - Self Healing
  - Automated Rollouts
  - Various other features like Service, Discovery and load balancing etc.

kubernetes

KUBERNETES CLUSTERS

- Containerized applications are deployed with K8s into highly available clusters.
- Clusters run over several computers called Worker Nodes, that are connected to work as a single unit.
- Containerized apps are automatically distributed among the Worke Nodes at deploy time.
- A master Nde manages the cluster-coordinating scheduling, scaling and rolling updates.

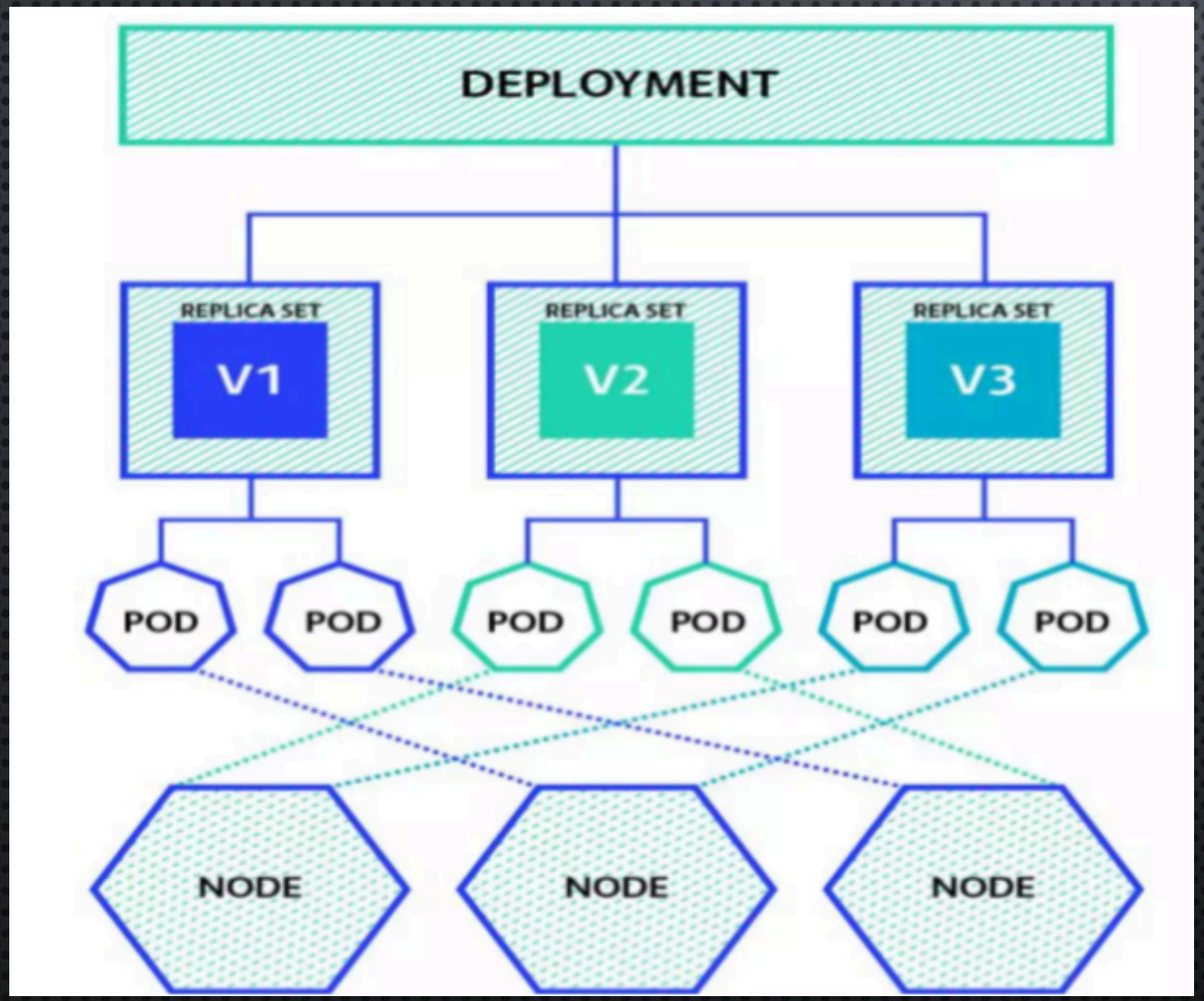
kubernetes

HOW DOSE K8S WORKS?

- Worker node (Slave)
  - This is where containers are deployed. These nodes contain:
    - Multiple Pods
    - Docker Engine
    - Kubelet – This is the most important component as it carries out the instructions from the master node.
- Master Node (Master)
    - This controls the deployment. This node contains:
    - API Server
    - Controller
    - Scheduler and Etcd - Handles configuration management, service, etc

kubernetes

KUBERNETES DEPLOYMENTS



- To run a containerized application in a cluster, a Deployment configuration is used:
  - The Deployment describes how K8s should create and update application instances.
  - The K8s master uses this to schedule the instances onto the work nodes.

kubernetes